COMP102 Lecture 6: Binary Search Trees

Search, Ordering Invariants, Height, and AVL Trees

Jalal Maaouni
April 27, 2026

UM6P University Mohammed VI Polytechnic
THE UM6P VANGUARD CENTER

#Empowering Minds.

Plan

1. Learning Outcomes
2. From Binary Trees to BSTs
3. Searching
4. Insertion and Sorted Traversal
5. Height and Complexity
6. Self-Balancing Trees
7. AVL Trees
8. Wrap-up

Learning Outcomes

Learning outcomes

By the end of this lecture, students should be able to:

- define the **binary search tree** property;
- check whether a binary tree is a valid BST;
- trace **search** and **insertion** in a BST;
- explain why **inorder traversal** of a BST returns sorted values;
- connect BST performance to the **height** of the tree;
- explain how bad insertion orders create **degenerate** BSTs;
- define the **AVL balance factor**;
- recognize the four AVL imbalance cases and the rotations used to repair them.

Last lecture: a binary tree was a **shape**. This lecture adds an **ordering invariant** so the shape can support efficient search.

From Binary Trees to BSTs

Continuity from last lecture

Continuity from last lecture

Binary tree

A node may have at most two children: **left** and **right**.

Continuity from last lecture

Binary tree

A node may have at most two children: **left** and **right**.

Problem

The words left and right do not yet tell us where a value should be.

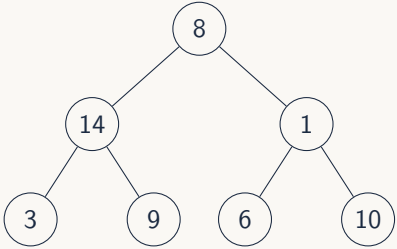
Continuity from last lecture

Binary tree

A node may have at most two children: **left** and **right**.

Problem

The words left and right do not yet tell us where a value should be.



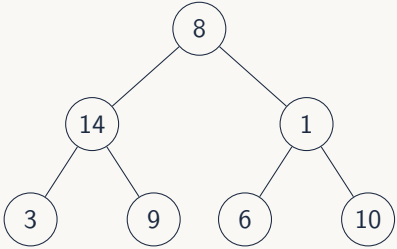
Continuity from last lecture

Binary tree

A node may have at most two children: **left** and **right**.

Problem

The words left and right do not yet tell us where a value should be.



A normal binary tree gives structure, but not search direction.

The binary search tree rule

The binary search tree rule

BST property

For every node:

$$\text{all values in left subtree} < \text{node value} < \text{all values in right subtree}$$

The binary search tree rule

BST property

For every node:

$$\text{all values in left subtree} < \text{node value} < \text{all values in right subtree}$$

Remark 2.3. Important

This rule must hold for the **whole subtree**, not only for the direct children.

The binary search tree rule

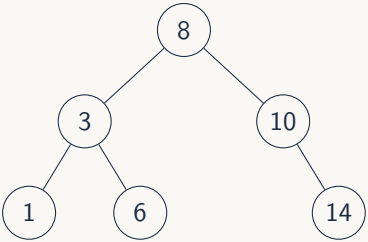
BST property

For every node:

$$\text{all values in left subtree} < \text{node value} < \text{all values in right subtree}$$

Remark 2.4. Important

This rule must hold for the **whole subtree**, not only for the direct children.



The binary search tree rule

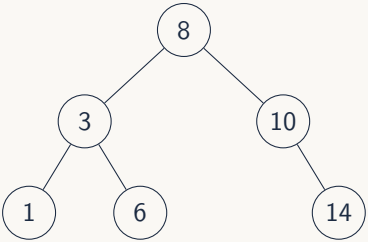
BST property

For every node:

$$\text{all values in left subtree} < \text{node value} < \text{all values in right subtree}$$

Remark 2.5. Important

This rule must hold for the **whole subtree**, not only for the direct children.



Same node class. Stronger meaning.

Same class, new invariant

Same class, new invariant

Python Node

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Same class, new invariant

Python `Node`

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Example 2.3. What changed?

The fields are identical to last lecture. The new part is a rule about where values are allowed to appear.

Same class, new invariant

Python `Node`

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Invariant

$$\text{left} < \text{node} < \text{right}$$

Example 2.4. What changed?

The fields are identical to last lecture. The new part is a rule about where values are allowed to appear.

Same class, new invariant

Python `Node`

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

Invariant

$$\text{left} < \text{node} < \text{right}$$

An invariant is a property that must remain true after every operation.

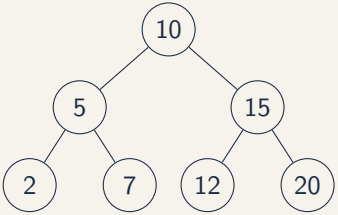
Example 2.5. What changed?

The fields are identical to last lecture. The new part is a rule about where values are allowed to appear.

Valid or invalid?

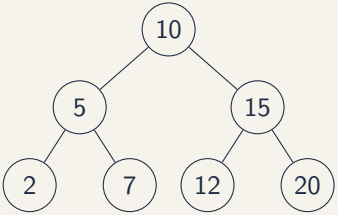
Valid or invalid?

valid?

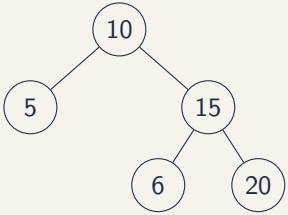


Valid or invalid?

valid?

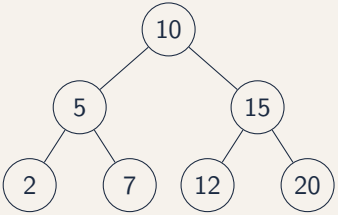


Valid?

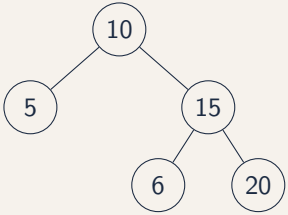


Valid or invalid?

valid?



Valid?



The value 6 is in the right subtree of 10, so it must be greater than 10. It is not. Therefore the tree is not a BST.

Searching

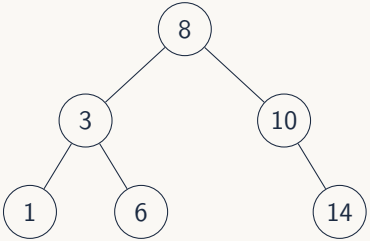
Search intuition

At each node, one comparison tells us which direction can still contain the target.

Search intuition

At each node, one comparison tells us which direction can still contain the target.

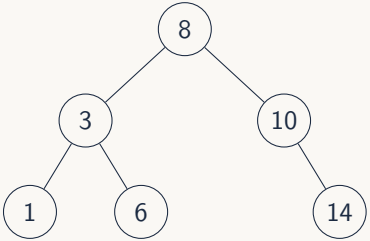
- target equals node $\rightarrow$ found



Search intuition

At each node, one comparison tells us which direction can still contain the target.

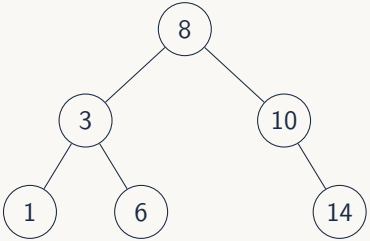
- target equals node $\rightarrow$ found
- target is smaller $\rightarrow$ go left



Search intuition

At each node, one comparison tells us which direction can still contain the target.

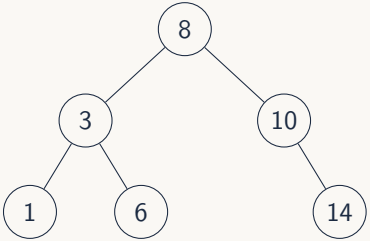
- target equals node $\rightarrow$ found
- target is smaller $\rightarrow$ go left
- target is larger $\rightarrow$ go right



Search intuition

At each node, one comparison tells us which direction can still contain the target.

- target equals node $\rightarrow$ found
- target is smaller $\rightarrow$ go left
- target is larger $\rightarrow$ go right



Remark 3.5. Key difference

In an ordinary binary tree, we may have to search both sides. In a BST, one side is ruled out.

BST search in Python

BST search in Python

Recursive search

```python
def search(root, target):
    if root is None:
        return False

    if target == root.value:
        return True
    elif target < root.value:
        return search(root.left, target)
    else:
        return search(root.right, target)
```

BST search in Python

Recursive search

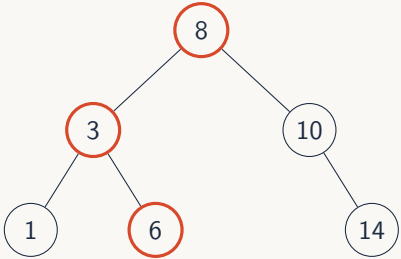
```python
def search(root, target):
    if root is None:
        return False

    if target == root.value:
        return True
    elif target < root.value:
        return search(root.left, target)
    else:
        return search(root.right, target)
```

The code is short because the BST invariant does most of the work.

Trace a search

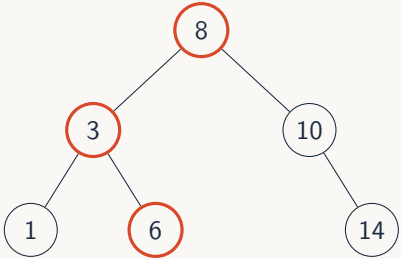
Search for **6**.



- Compare with 8: $6 < 8$, go left.

Trace a search

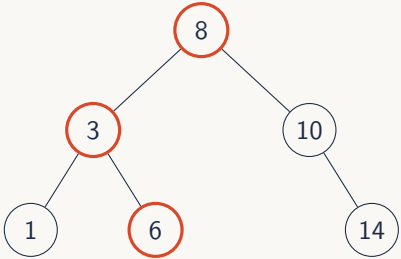
Search for **6**.



- Compare with 8: $6 < 8$, go left.
- Compare with 3: $6 > 3$, go right.

Trace a search

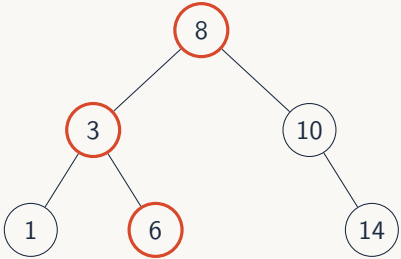
Search for **6**.



- Compare with 8: $6 < 8$, go left.
- Compare with 3: $6 > 3$, go right.
- Compare with 6: found.

Trace a search

Search for **6**.



- Compare with 8: $6 < 8$, go left.
- Compare with 3: $6 > 3$, go right.
- Compare with 6: found.

Path

$8 \rightarrow 3 \rightarrow 6$

Insertion and Sorted Traversal

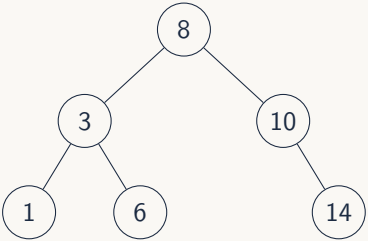
Insertion intuition

Insertion is search for the correct empty position.

Insertion intuition

Insertion is search for the correct empty position.

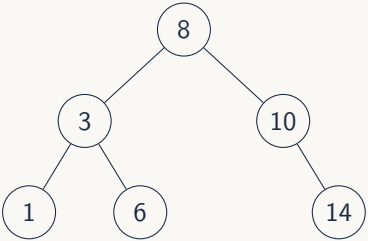
- Start at the root.



Insertion intuition

Insertion is search for the correct empty position.

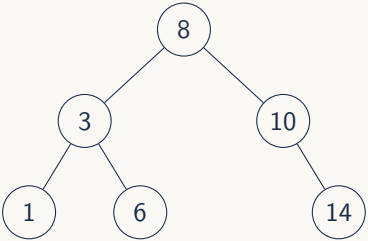
- Start at the root.
- Move left or right using the BST rule.



Insertion intuition

Insertion is search for the correct empty position.

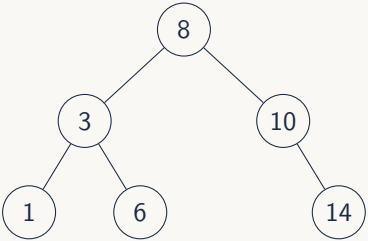
- Start at the root.
- Move left or right using the BST rule.
- Stop when the required child pointer is empty.



Insertion intuition

Insertion is search for the correct empty position.

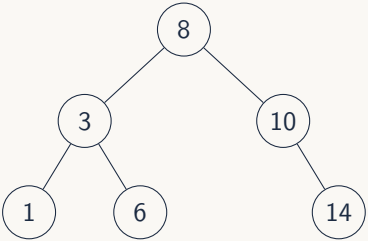
- Start at the root.
- Move left or right using the BST rule.
- Stop when the required child pointer is empty.
- Attach the new node there.



Insertion intuition

Insertion is search for the correct empty position.

- Start at the root.
- Move left or right using the BST rule.
- Stop when the required child pointer is empty.
- Attach the new node there.



Remark 4.6. For this lecture

We will assume duplicate values are not inserted.

BST insertion in Python

BST insertion in Python

Recursive insertion

```python
def insert(root, value):
    if root is None:
        return Node(value)

    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)

    return root
```

BST insertion in Python

Recursive insertion

```python
def insert(root, value):
    if root is None:
        return Node(value)

    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)

    return root
```

Each recursive call moves into the only subtree where the new value is allowed to appear.

Build a BST step by step

Insert the values:

$$8,\ 3,\ 10,\ 1,\ 6,\ 14$$

Build a BST step by step

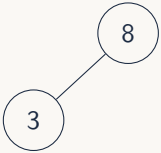
Insert the values:

$$8,\ 3,\ 10,\ 1,\ 6,\ 14$$

Build a BST step by step

Insert the values:

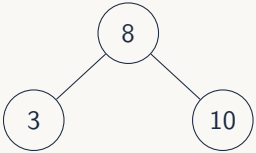
8, 3, 10, 1, 6, 14



Build a BST step by step

Insert the values:

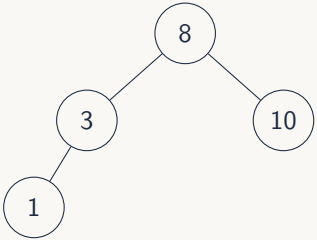
8, 3, 10, 1, 6, 14



Build a BST step by step

Insert the values:

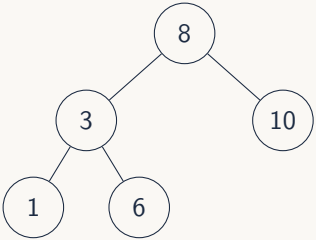
8, 3, 10, 1, 6, 14



Build a BST step by step

Insert the values:

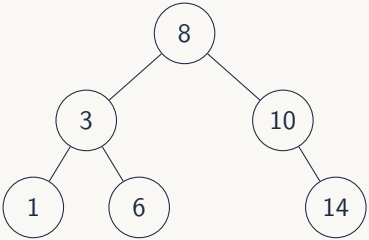
8, 3, 10, 1, 6, 14



Build a BST step by step

Insert the values:

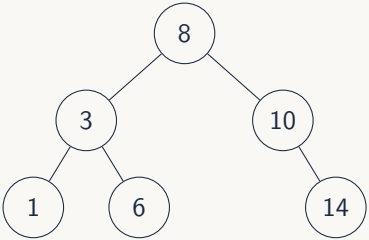
8, 3, 10, 1, 6, 14



Build a BST step by step

Insert the values:

$$8,\ 3,\ 10,\ 1,\ 6,\ 14$$



The final shape depends on the insertion order.

Inorder becomes sorted order

From last lecture:

$$\text{inorder} = \text{Left, Node, Right}$$

Inorder becomes sorted order

From last lecture:

$$\text{inorder} = \text{Left, Node, Right}$$

For a BST:

- left subtree values are smaller;
- the node is in the middle;
- right subtree values are larger.

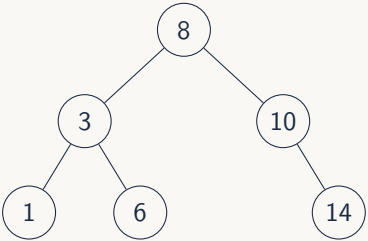
Inorder becomes sorted order

From last lecture:

$$\text{inorder} = \text{Left, Node, Right}$$

For a BST:

- left subtree values are smaller;
- the node is in the middle;
- right subtree values are larger.



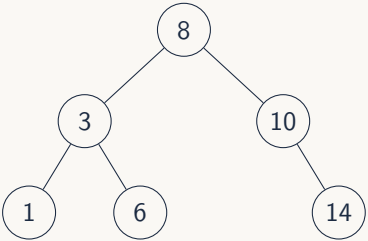
Inorder becomes sorted order

From last lecture:

$$\text{inorder} = \text{Left, Node, Right}$$

For a BST:

- left subtree values are smaller;
- the node is in the middle;
- right subtree values are larger.



Output

1, 3, 6, 8, 10, 14

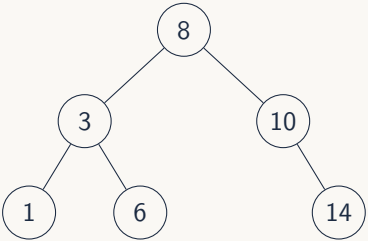
Inorder becomes sorted order

From last lecture:

$$\text{inorder} = \text{Left, Node, Right}$$

For a BST:

- left subtree values are smaller;
- the node is in the middle;
- right subtree values are larger.



Output

1, 3, 6, 8, 10, 14

The traversal did not change. The meaning changed because the tree now has an ordering invariant.

Minimum and maximum

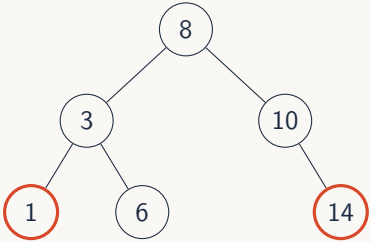
The BST rule makes two operations especially simple.

Minimum and maximum

The BST rule makes two operations especially simple.

Minimum

Keep following **left** pointers until there is no left child.



Minimum and maximum

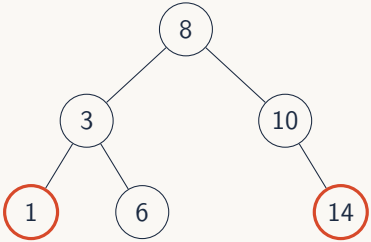
The BST rule makes two operations especially simple.

Minimum

Keep following **left** pointers until there is no left child.

Maximum

Keep following **right** pointers until there is no right child.



Minimum and maximum

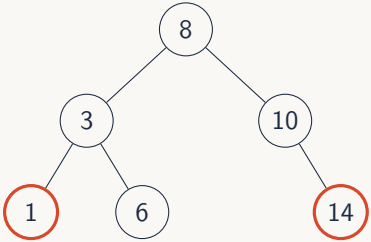
The BST rule makes two operations especially simple.

Minimum

Keep following **left** pointers until there is no left child.

Maximum

Keep following **right** pointers until there is no right child.



These operations still depend on height: they follow one path from the root.

Height and Complexity

Cost depends on height

Let h be the height of the tree.

Operation	**Cost**
Search	$O(h)$
Insert	$O(h)$
Minimum / maximum	$O(h)$
Inorder traversal	$O(n)$

Cost depends on height

Let h be the height of the tree.

Operation	**Cost**
Search	$O(h)$
Insert	$O(h)$
Minimum / maximum	$O(h)$
Inorder traversal	$O(n)$

Remark 5.2. Main question

How large can h become?

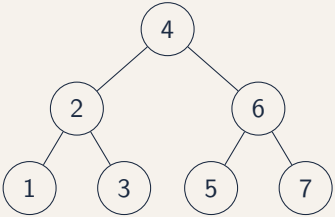
Balanced vs degenerate shape

Same values. Different height.

Balanced vs degenerate shape

Same values. Different height.

Balanced shape

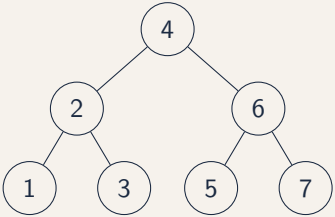


height $= 2$

Balanced vs degenerate shape

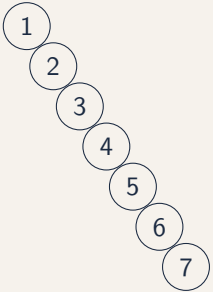
Same values. Different height.

Balanced shape



height $= 2$

Degenerate shape

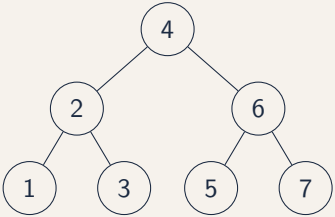


height $= 6$

Balanced vs degenerate shape

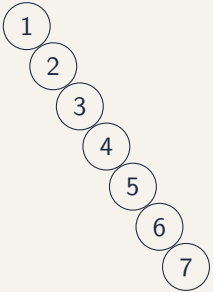
Same values. Different height.

Balanced shape



height $= 2$

Degenerate shape



height $= 6$

The BST rule gives search direction, but it does not guarantee a good shape.

The bad insertion order

Insert sorted values:

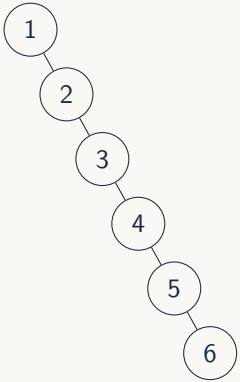
$$1,\ 2,\ 3,\ 4,\ 5,\ 6$$

The bad insertion order

Insert sorted values:

$$1,\ 2,\ 3,\ 4,\ 5,\ 6$$

- Every new value is larger than all previous values.

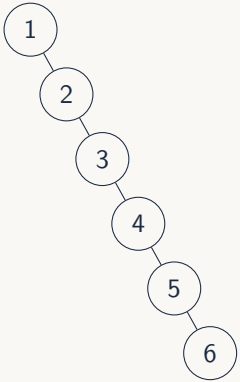


The bad insertion order

Insert sorted values:

$$1,\ 2,\ 3,\ 4,\ 5,\ 6$$

- Every new value is larger than all previous values.
- Every insertion goes to the right.

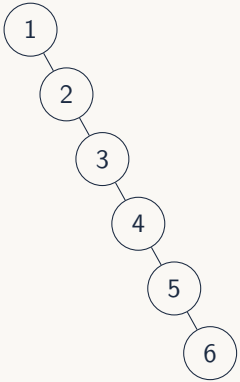


The bad insertion order

Insert sorted values:

$$1,\ 2,\ 3,\ 4,\ 5,\ 6$$

- Every new value is larger than all previous values.
- Every insertion goes to the right.
- The BST becomes a linked list in disguise.

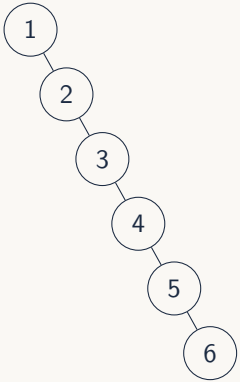


The bad insertion order

Insert sorted values:

$$1,\ 2,\ 3,\ 4,\ 5,\ 6$$

- Every new value is larger than all previous values.
- Every insertion goes to the right.
- The BST becomes a linked list in disguise.



Worst-case BST search is $O(n)$ because the height can become $n - 1$.

Best case, worst case

Shape	**Height**	**Search / insert**
Balanced BST	$O(\log n)$	$O(\log n)$
Degenerate BST	$O(n)$	$O(n)$

Best case, worst case

Shape	**Height**	**Search / insert**
Balanced BST	$O(\log n)$	$O(\log n)$
Degenerate BST	$O(n)$	$O(n)$

A BST is only efficient if the height stays small.

Best case, worst case

Shape	Height	Search / insert
Balanced BST	$O(\log n)$	$O(\log n)$
Degenerate BST	$O(n)$	$O(n)$

A BST is only efficient if the height stays small.

Remark 5.5. Transition

Self-balancing trees add one more promise: after updates, they repair the shape.

Self-Balancing Trees

Self-balancing trees

Self-balancing trees

Problem

A normal BST can become tall depending on insertion order.

Self-balancing trees

Problem

A normal BST can become tall depending on insertion order.

Idea

A self-balancing BST keeps the ordering rule, but also controls the height.

Self-balancing trees

Problem

A normal BST can become tall depending on insertion order.

Idea

A self-balancing BST keeps the ordering rule, but also controls the height.

Examples include:

- AVL trees
- red-black trees
- B-trees

Self-balancing trees

Problem

A normal BST can become tall depending on insertion order.

Idea

A self-balancing BST keeps the ordering rule, but also controls the height.

Examples include:

- AVL trees
- red-black trees
- B-trees

In this lecture, AVL trees are the only concrete example.

AVL Trees

AVL trees: definition

AVL trees: definition

Balance factor

For any node:

$$\text{balance factor} = \text{height(left subtree)} - \text{height(right subtree)}$$

AVL trees: definition

Balance factor

For any node:

$$\text{balance factor} = \text{height(left subtree)} - \text{height(right subtree)}$$

AVL condition

Every node must have balance factor:

$$-1,\ 0,\ \text{or } 1$$

AVL trees: definition

Balance factor

For any node:

$$\text{balance factor} = \text{height(left subtree)} - \text{height(right subtree)}$$

AVL condition

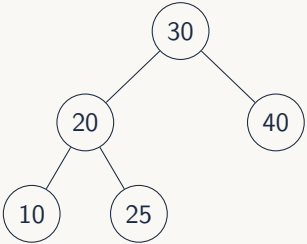
Every node must have balance factor:

$$-1,\ 0,\ \text{or } 1$$

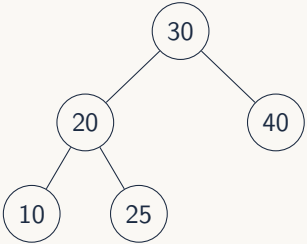
An AVL tree is a BST that refuses to become too left-heavy or too right-heavy at any node.

Compute balance factors

Compute balance factors

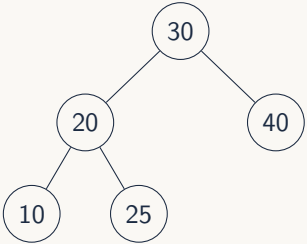


Compute balance factors



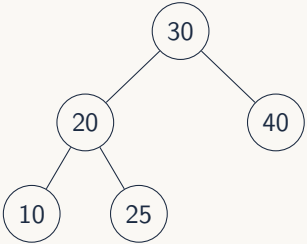
- leaves have height 0;

Compute balance factors



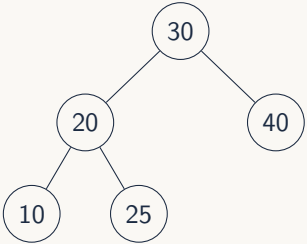
- leaves have height 0;
- node 20 has left height 0 and right height 0;

Compute balance factors



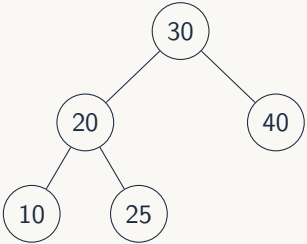
- leaves have height 0;
- node 20 has left height 0 and right height 0;
- root 30 has left height 1 and right height 0;

Compute balance factors



- leaves have height 0;
- node 20 has left height 0 and right height 0;
- root 30 has left height 1 and right height 0;
- balance factor at 30 is $1 - 0 = 1$.

Compute balance factors

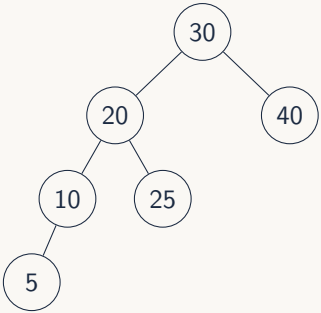


- leaves have height 0;
- node 20 has left height 0 and right height 0;
- root 30 has left height 1 and right height 0;
- balance factor at 30 is $1 - 0 = 1$.

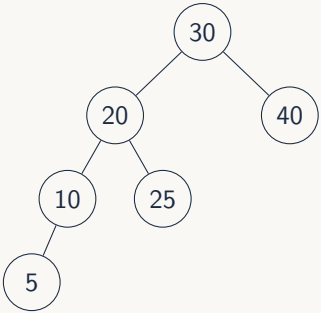
This tree is AVL-balanced because every balance factor is in $\{-1, 0, 1\}$.

When a node is unbalanced

When a node is unbalanced

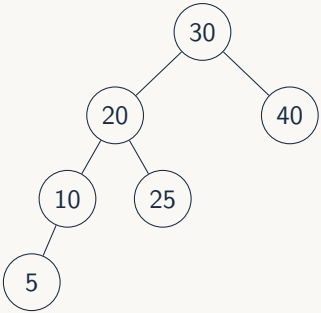


When a node is unbalanced



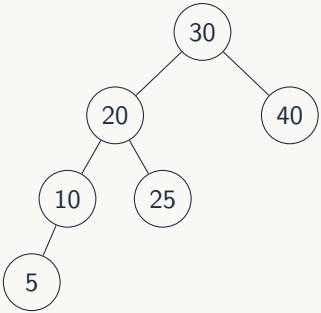
- left subtree of 30 has height 2;

When a node is unbalanced



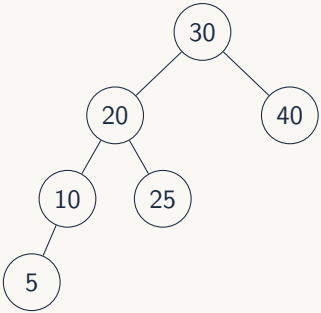
- left subtree of 30 has height 2;
- right subtree of 30 has height 0;

When a node is unbalanced



- left subtree of 30 has height 2;
- right subtree of 30 has height 0;
- balance factor at 30 is $2 - 0 = 2$.

When a node is unbalanced



- left subtree of 30 has height 2;
- right subtree of 30 has height 0;
- balance factor at 30 is $2 - 0 = 2$.

Remark 7.6. Conclusion

Balance factor 2 is not allowed in an AVL tree.

Rotations repair local imbalance

Rotations repair local imbalance

Rotation

A rotation is a local rewrite of a small part of the tree.

Rotations repair local imbalance

Rotation

A rotation is a local rewrite of a small part of the tree.

A rotation must preserve:

- the BST ordering property;
- the same set of values;
- a smaller local height.

Rotations repair local imbalance

Rotation

A rotation is a local rewrite of a small part of the tree.

A rotation must preserve:

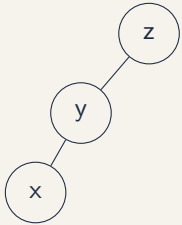
- the BST ordering property;
- the same set of values;
- a smaller local height.

Rotations are not arbitrary rearrangements. They are controlled repairs that keep the inorder order unchanged.

Right rotation: Left-Left case

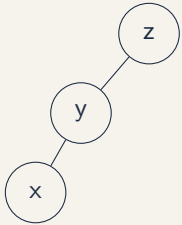
Right rotation: Left-Left case

Before



Right rotation: Left-Left case

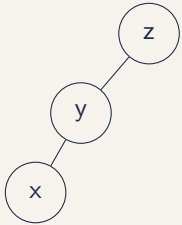
Before



$\Rightarrow$

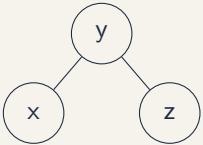
Right rotation: Left-Left case

Before



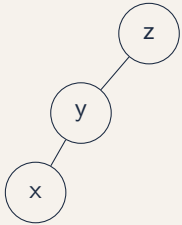
$\Rightarrow$

After



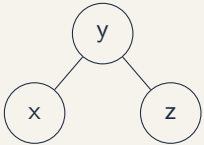
Right rotation: Left-Left case

Before



$\Rightarrow$

After

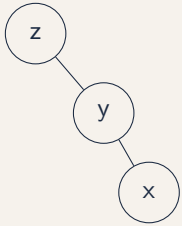


A Left-Left imbalance is repaired by a **right rotation**.

Left rotation: Right-Right case

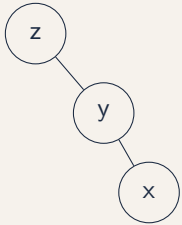
Left rotation: Right-Right case

Before



Left rotation: Right-Right case

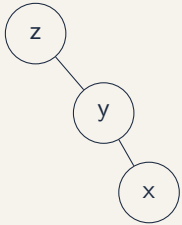
Before



$\Rightarrow$

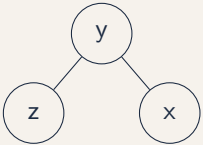
Left rotation: Right-Right case

Before



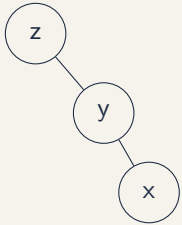
$\Rightarrow$

After



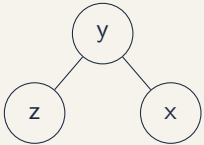
Left rotation: Right-Right case

Before



$\Rightarrow$

After

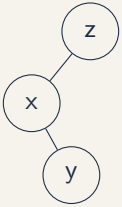


A Right-Right imbalance is repaired by a **left rotation**.

Double rotation: Left-Right case

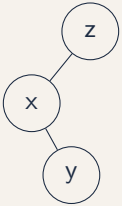
Double rotation: Left-Right case

Left-Right imbalance

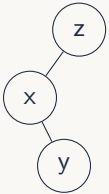


Double rotation: Left-Right case

Left-Right imbalance

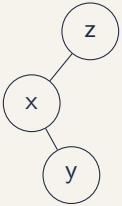


Before

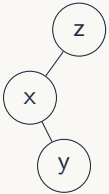


Double rotation: Left-Right case

Left-Right imbalance

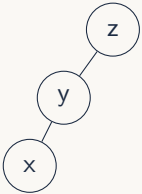


Before



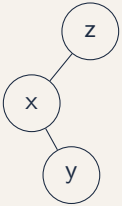
$\Rightarrow$

1. Rotate left at x

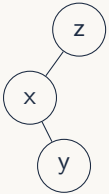


Double rotation: Left-Right case

Left-Right imbalance

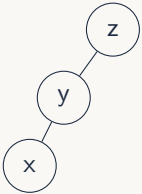


Before



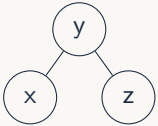
$\Rightarrow$

1. Rotate left at x



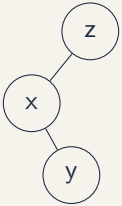
$\Rightarrow$

2. Rotate right at z

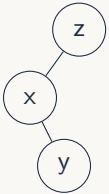


Double rotation: Left-Right case

Left-Right imbalance

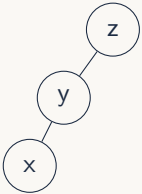


Before



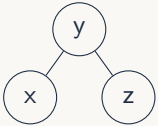
$\Rightarrow$

1. Rotate left at x



$\Rightarrow$

2. Rotate right at z

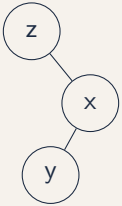


A Left-Right case is first converted into a Left-Left case, then repaired by a right rotation.

Double rotation: Right-Left case

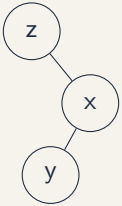
Double rotation: Right-Left case

Right-Left imbalance

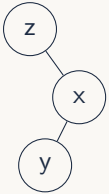


Double rotation: Right-Left case

Right-Left imbalance

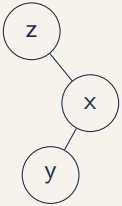


Before

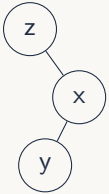


Double rotation: Right-Left case

Right-Left imbalance

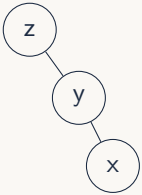


Before



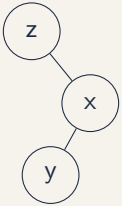
$\Rightarrow$

1. Rotate right at x

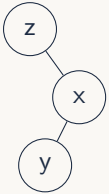


Double rotation: Right-Left case

Right-Left imbalance

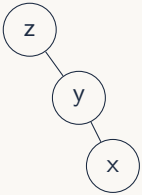


Before



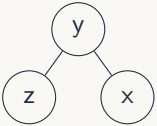
$\Rightarrow$

1. Rotate right at x



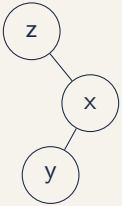
$\Rightarrow$

2. Rotate left at z

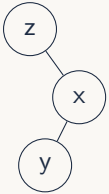


Double rotation: Right-Left case

Right-Left imbalance

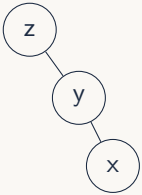


Before



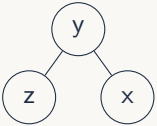
$\Rightarrow$

1. Rotate right at x



$\Rightarrow$

2. Rotate left at z



A Right-Left case is first converted into a Right-Right case, then repaired by a left rotation.

The four AVL insertion cases

Case	Shape	Repair
Left-Left	left child of left subtree	right rotation
Right-Right	right child of right subtree	left rotation
Left-Right	right child of left subtree	left rotation, then right rotation
Right-Left	left child of right subtree	right rotation, then left rotation

The four AVL insertion cases

Case	Shape	Repair
Left-Left	left child of left subtree	right rotation
Right-Right	right child of right subtree	left rotation
Left-Right	right child of left subtree	left rotation, then right rotation
Right-Left	left child of right subtree	right rotation, then left rotation

After insertion, AVL rebalancing repairs the first ancestor whose balance factor becomes invalid.

AVL insertion: high-level algorithm

AVL insertion: high-level algorithm

1. Insert the value exactly as in a normal BST.

AVL insertion: high-level algorithm

1. Insert the value exactly as in a normal BST.
2. Walk back upward toward the root.

AVL insertion: high-level algorithm

1. Insert the value exactly as in a normal BST.
2. Walk back upward toward the root.
3. Update heights.

AVL insertion: high-level algorithm

1. Insert the value exactly as in a normal BST.
2. Walk back upward toward the root.
3. Update heights.
4. Compute balance factors.

AVL insertion: high-level algorithm

1. Insert the value exactly as in a normal BST.
2. Walk back upward toward the root.
3. Update heights.
4. Compute balance factors.
5. If a node becomes unbalanced, apply the appropriate rotation.

AVL insertion: high-level algorithm

1. Insert the value exactly as in a normal BST.
2. Walk back upward toward the root.
3. Update heights.
4. Compute balance factors.
5. If a node becomes unbalanced, apply the appropriate rotation.

The BST rule decides *where* the value goes. The AVL rule decides whether the tree needs repair afterward.

Deleting from a BST

Deleting from a BST

Deletion

Deleting a value from a BST means removing a node while preserving the BST ordering property.

Deleting from a BST

Deletion

Deleting a value from a BST means removing a node while preserving the BST ordering property.

There are three cases:

1. the node is a leaf;
2. the node has one child;
3. the node has two children.

Deleting from a BST

Deletion

Deleting a value from a BST means removing a node while preserving the BST ordering property.

There are three cases:

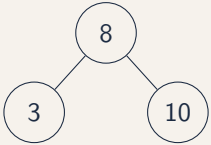
1. the node is a leaf;
2. the node has one child;
3. the node has two children.

The first two cases are simple. The third case needs a replacement value.

Deletion case 1: leaf node

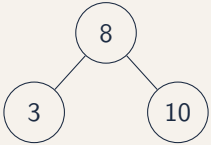
Deletion case 1: leaf node

Before



Deletion case 1: leaf node

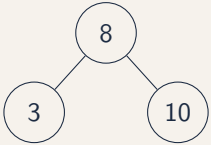
Before



$\Rightarrow$

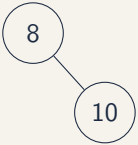
Deletion case 1: leaf node

Before



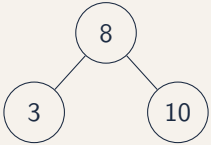
$\Rightarrow$

After deleting 3



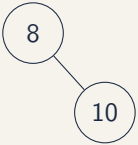
Deletion case 1: leaf node

Before



$\Rightarrow$

After deleting 3

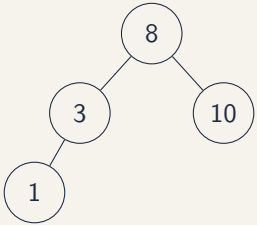


If the node has no children, simply remove it.

Deletion case 2: one child

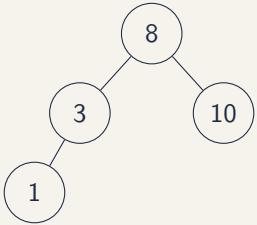
Deletion case 2: one child

Before



Deletion case 2: one child

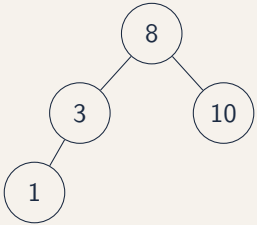
Before



$\Rightarrow$

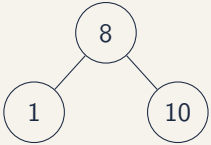
Deletion case 2: one child

Before



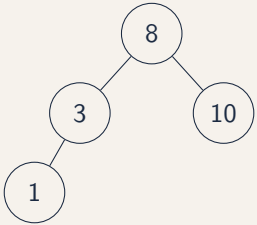
$\Rightarrow$

After deleting 3



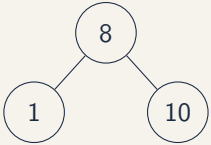
Deletion case 2: one child

Before



$\Rightarrow$

After deleting 3



If the node has one child, connect the parent directly to that child.

Deletion case 3: two children

Deletion case 3: two children

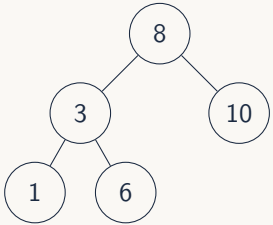
Problem

If the node has two children, removing it leaves two subtrees that must still fit the BST order.

Deletion case 3: two children

Problem

If the node has two children, removing it leaves two subtrees that must still fit the BST order.

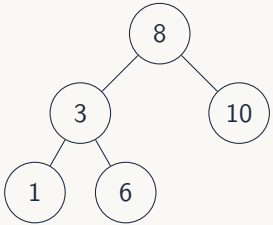


Delete 3

Deletion case 3: two children

Problem

If the node has two children, removing it leaves two subtrees that must still fit the BST order.



Delete 3

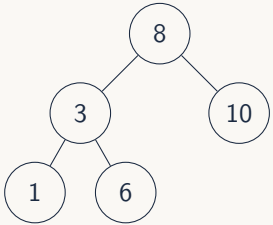
We replace it with either:

- its **inorder predecessor**; or
- its **inorder successor**.

Deletion case 3: two children

Problem

If the node has two children, removing it leaves two subtrees that must still fit the BST order.



Delete 3

We replace it with either:

- its **inorder predecessor**; or
- its **inorder successor**.

Common choice: use the inorder successor, the smallest value in the right subtree.

Inorder successor

Inorder successor

Definition

The inorder successor of a node is the next value after it in sorted order.

Inorder successor

Definition

The inorder successor of a node is the next value after it in sorted order.

For a node with a right subtree:

$$\text{successor} = \text{minimum value in the right subtree}$$

Inorder successor

Definition

The inorder successor of a node is the next value after it in sorted order.

For a node with a right subtree:

$$\text{successor} = \text{minimum value in the right subtree}$$

To find it:

1. go once to the right;
2. then keep going left.

Inorder successor

Definition

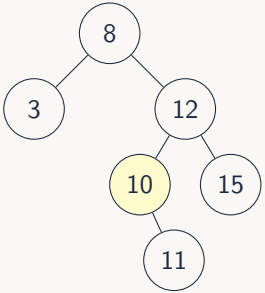
The inorder successor of a node is the next value after it in sorted order.

For a node with a right subtree:

successor = minimum value in the right subtree

To find it:

1. go once to the right;
2. then keep going left.

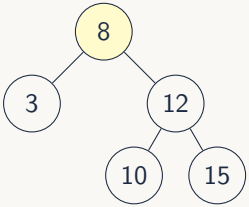


Successor of 8 is 10.

Deleting a node with two children

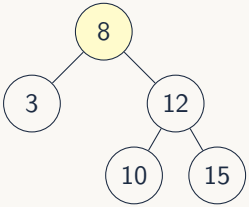
Deleting a node with two children

Before



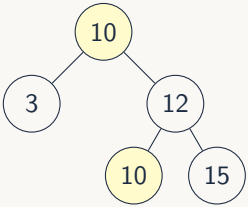
Deleting a node with two children

Before



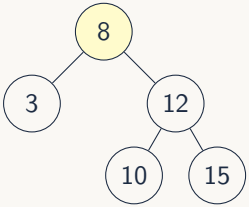
$\Rightarrow$

Copy successor



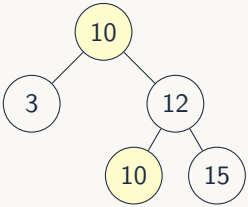
Deleting a node with two children

Before



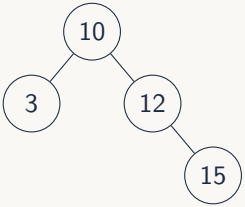
$\Rightarrow$

Copy successor

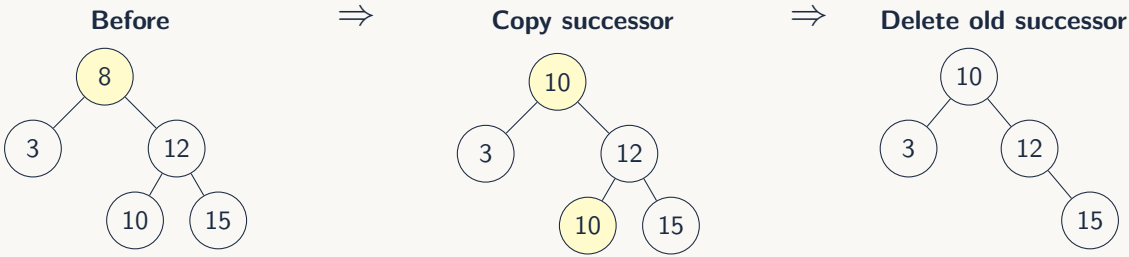


$\Rightarrow$

Delete old successor



Deleting a node with two children



Two-child deletion becomes simpler by replacing the node with its inorder successor, then deleting that successor from its old position.

BST deletion algorithm

BST deletion algorithm

Recursive idea

First search for the value exactly as in normal BST search.

BST deletion algorithm

Recursive idea

First search for the value exactly as in normal BST search.

When the node is found:

BST deletion algorithm

Recursive idea

First search for the value exactly as in normal BST search.

When the node is found:

1. If it has no child, return `None`.

BST deletion algorithm

Recursive idea

First search for the value exactly as in normal BST search.

When the node is found:

1. If it has no child, return `None`.
2. If it has one child, return that child.

BST deletion algorithm

Recursive idea

First search for the value exactly as in normal BST search.

When the node is found:

1. If it has no child, return `None`.
2. If it has one child, return that child.
3. If it has two children:
   - 3.1 find the inorder successor;
   - 3.2 copy its value into the current node;
   - 3.3 delete the successor from the right subtree.

BST deletion algorithm

Recursive idea

First search for the value exactly as in normal BST search.

When the node is found:

1. If it has no child, return `None`.
2. If it has one child, return that child.
3. If it has two children:
   - 3.1 find the inorder successor;
   - 3.2 copy its value into the current node;
   - 3.3 delete the successor from the right subtree.

Deletion preserves the BST property by replacing a removed value with the next valid sorted value.

Wrap-up

Comparison

Structure	Rule	Search
Binary tree	no ordering rule	$O(n)$
BST	left $<$ node $<$ right	$O(h)$
AVL tree	BST rule $+$ controlled height	$O(\log n)$

Comparison

Structure	Rule	Search
Binary tree	no ordering rule	$O(n)$
BST	left $<$ node $<$ right	$O(h)$
AVL tree	BST rule $+$ controlled height	$O(\log n)$

The main progression is: structure $\rightarrow$ order $\rightarrow$ balance.

Takeaways

Takeaways

- A BST is a binary tree with an **ordering invariant**.

Takeaways

- A BST is a binary tree with an **ordering invariant**.
- Search and insertion follow that invariant one comparison at a time.

Takeaways

- A BST is a binary tree with an **ordering invariant**.
- Search and insertion follow that invariant one comparison at a time.
- Inorder traversal of a BST returns values in sorted order.

Takeaways

- A BST is a binary tree with an **ordering invariant**.
- Search and insertion follow that invariant one comparison at a time.
- Inorder traversal of a BST returns values in sorted order.
- BST performance depends on height.

Takeaways

- A BST is a binary tree with an **ordering invariant**.
- Search and insertion follow that invariant one comparison at a time.
- Inorder traversal of a BST returns values in sorted order.
- BST performance depends on height.
- A bad insertion order can make a BST degenerate.

Takeaways

- A BST is a binary tree with an **ordering invariant**.
- Search and insertion follow that invariant one comparison at a time.
- Inorder traversal of a BST returns values in sorted order.
- BST performance depends on height.
- A bad insertion order can make a BST degenerate.
- AVL trees keep height controlled using balance factors and rotations.

Takeaways

- A BST is a binary tree with an **ordering invariant**.
- Search and insertion follow that invariant one comparison at a time.
- Inorder traversal of a BST returns values in sorted order.
- BST performance depends on height.
- A bad insertion order can make a BST degenerate.
- AVL trees keep height controlled using balance factors and rotations.

A binary search tree gives us direction. A self-balancing tree keeps that direction efficient.